Vector Embeddings & Vector Databases

for RAG, GenAI & Agentic AI applications

Hand-written notes
intuition -> maths -> code

"I love pizza" =
[0.12, -0.55, 0.91 ...]

Written by Chandan Chaudhari
GitHub: github.com/chandanc5525

What's inside

1. The Big Picture: Why This Matters ..... p. 2
2. Embeddings: The Intuition ..... p. 3
3. What Exactly Is a Vector Embedding? ..... p. 4
4. Similarity: Measuring "Closeness" ..... p. 6
5. Choosing an Embedding Model ..... p. 8
6. RAG Architecture, End to End ..... p. 13
7. Chunking: Cutting Documents Wisely ..... p. 14
8. Vector Databases: Why and What ..... p. 16
9. How ANN Indexes Work ..... p. 18
10. Hands-On: Chroma + Metadata Filters ..... p. 21
11. Full RAG Pipeline in Code ..... p. 23
12. Making Retrieval Better ..... p. 25
13. Agentic RAG: Retrieval as a Tool ..... p. 28
14. Evaluating Your RAG System ..... p. 30
15. Mistakes, Checklist & Cheat Sheet ..... p. 31

The Big Picture: Why This Matters

LLMs like Claude or GPT are brilliant but have 4 weaknesses:

- Knowledge cutoff - they don't know what happened after training.
- No private data - they never saw your company PDFs, wiki, tickets, emails.
- Hallucination - when unsure they confidently make things up.
- Limited context window - you cannot paste 10,000 pages in every prompt.

RAG (Retrieval-Augmented Generation) fixes this: first FIND the few relevant pieces of text, then hand only those to the LLM along with the question. Think of it as an open-book exam: the LLM is the student, the retrieved chunks are the pages of the book it is allowed to open. But how do we FIND the relevant text? Keyword search fails when words differ ("car" vs "automobile"). We need search by meaning - and that is exactly what vector embeddings + vector databases give us.

One-line summary

Embedding = meaning of text converted into a list of numbers.
Vector database = a database built to quickly find the numbers closest to a given list of numbers.
RAG = retrieve closest chunks + give them to the LLM to answer.

Roadmap of these notes

Chapters	Topic	Question it answers
2 - 3	Embeddings: intuition + definition	How can numbers represent meaning?
4 - 5	Similarity maths + models (incl. Hugging Face)	How do I measure closeness? Which model?
6 - 7	RAG architecture + chunking	How do I prepare and use my documents?
8 - 9	Vector DBs + ANN indexes	How to search millions of vectors fast?
10 - 11	Code: Chroma, FAISS, full RAG	How do I build it?
12 - 13	Better retrieval + Agentic RAG	How do I make it production-grade and smart?
14 - 15	Evaluation, mistakes, cheat sheet	How do I know it works?

Embeddings: The Intuition

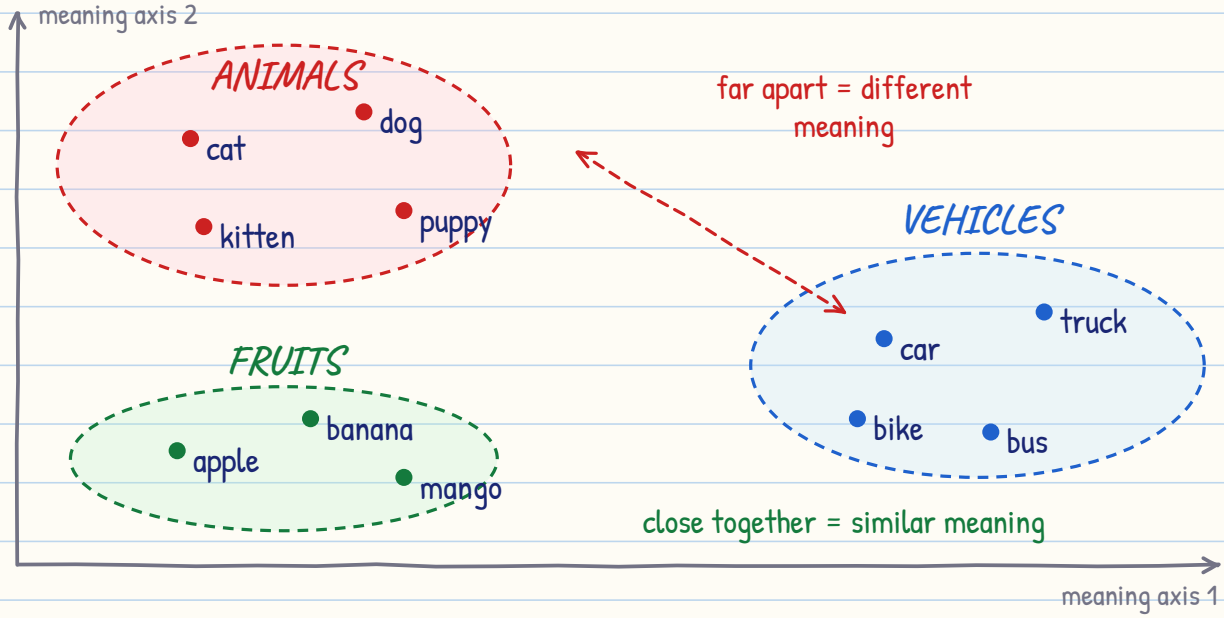
Story 1: the smart librarian

A normal library arranges books alphabetically. Finding "books about dogs" means checking many shelves. Now imagine a librarian who arranges books by meaning: dog books next to puppy books next to wolf books; car books on the other side of the building. To find what you want, you just walk to the right spot and look nearby.

An embedding gives every piece of text an address in this "meaning-space". Close addresses = similar meaning.

Story 2: GPS coordinates

Thane is at about (19.2, 72.9) - two numbers (latitude, longitude) tell you where it is, and two cities with close numbers are physically close. An embedding does the same for meaning, but needs hundreds of numbers instead of two, because meaning has far more directions than a flat map.



(Above I squashed it to 2 dimensions so we can draw it. Real embeddings have 384, 768, 1536 or more dimensions - impossible to draw, but the idea is identical.)

Why so many dimensions?

Each dimension captures some hidden aspect of meaning - topic, sentiment, formality, "is it about animals?", "is it a question?" ... The model learns these on its own; we cannot name

most of them. More dimensions = room to tell more subtle meanings apart (but costs more memory + time).

Famous example: word arithmetic

In classic word embeddings (word2vec) directions carry meaning:

`vector("king") - vector("man") + vector("woman") ~ vector("queen")`

So "male → female" is a direction in space. This shows embeddings capture relationships, not just spelling.

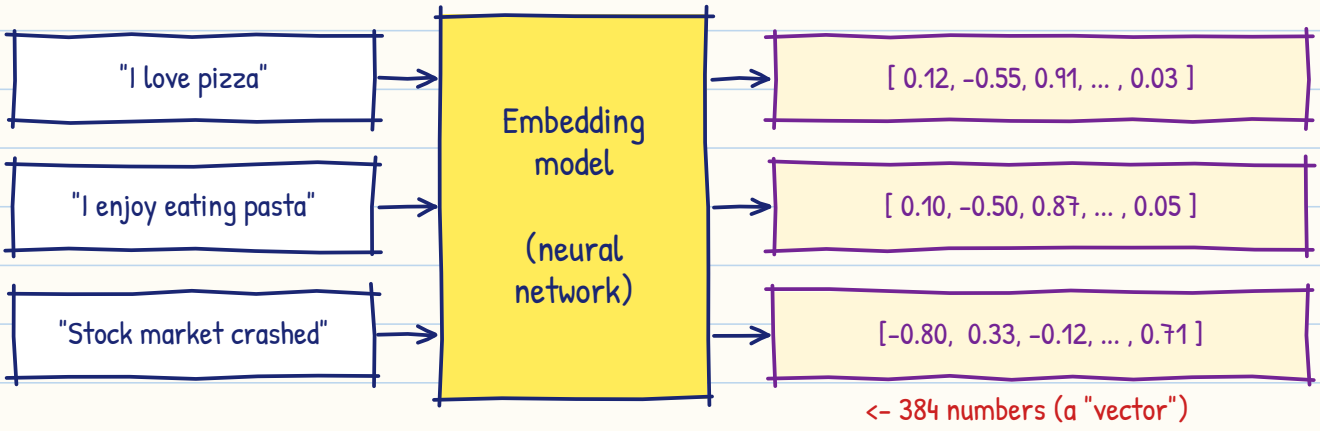
Two properties to remember

- Similar meaning → nearby vectors (even with different words: "buy a car" ~ "purchase an automobile").
- Different meaning → far apart vectors.

Search becomes a geometry problem: "find the nearest points to my question".

What Exactly Is a Vector Embedding?

A vector is just an ordered list of numbers, e.g. *[0.12, -0.55, 0.91]*. A vector embedding is such a list produced by a neural network (the embedding model) from some input: a word, sentence, paragraph, image, audio, or code.



- Fixed length: a 3-word sentence and a 300-word paragraph both become, say, 384 numbers.
- Deterministic: same text + same model = same vector.
- Model-specific: numbers from model A mean nothing to model B. Never mix them!

- Dense: almost every number is non-zero (unlike keyword vectors that are mostly zeros).

How is the embedding model made? (simple version)

1. Tokenize: text is split into tokens (word pieces).
2. Transformer reads all tokens and produces one vector per token, using context ("bank" of a river vs "bank" for money get different vectors).
3. Pooling: average (or take a special token) to get ONE vector for the whole text.
4. Contrastive training: show millions of pairs, e.g. (question, correct answer). Pull their vectors CLOSE, push random mismatched pairs FAR. After training, similar meaning ends up nearby.

Kinds of embeddings

Type	Input	Used for
Text	sentences, passages	RAG, semantic search, clustering, dedupe
Image	pictures	image search, product similarity
Multimodal (CLIP-like)	text + images in same space	search photos using words
Code	functions, files	code search, coding agents
Audio	speech / sound	voice search, music matching

Code: make your first embeddings

```python
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")  # small, 384 dims, CPU ok

sentences = [
    "The cat sat on the mat",
    "A kitten is resting on a rug",
    "Stock markets fell sharply today",
]
vecs = model.encode(sentences)      # numpy array, shape (3, 384)
print(vecs.shape)                   # (3, 384)
print(vecs[0][:5])                  # first 5 numbers of sentence 0
```

Watch out: embeddings capture "aboutness", not truth

"I love cats" and "I hate cats" are about the same topic, so their vectors can be very close even though the meaning is opposite. Embeddings find related text, not necessarily agreeing text. Exact things like product codes, names, numbers are also weak → fix later with hybrid search.

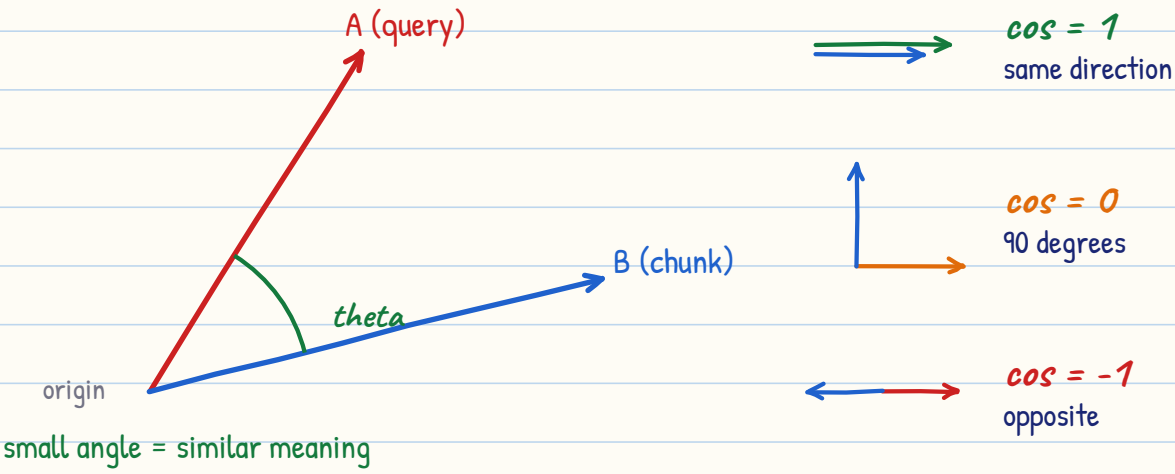
Golden rule

Use the SAME embedding model for your documents and for your queries. Different model or different dimension = garbage results (or an error).

Similarity: Measuring "Closeness"

Once text is a vector, "how similar are two texts?" becomes "how close are two vectors?".

Three common measures:



small angle = similar meaning

1) Cosine similarity - the favourite

Looks only at the angle between vectors, ignoring their length. Range: -1 to +1.

$$\cos(\theta) = (A \cdot B) / (|A| \times |B|)$$

$A \cdot B$ = dot product = multiply matching numbers and add. $|A|$ = length of A = square root of the sum of squares.

Worked example. A = [1, 2, 3], B = [2, 4, 6]. Dot = 1x2 + 2x4 + 3x6 = 28. |A| = sqrt(14), |B| =

sqrt(56), product = sqrt(784) = 28. So cos = 28/28 = 1.0: same direction, B is just twice as long. Cosine says "identical meaning" - perfect for text, since a long and a short text on one topic point the same way.

2) Dot product

Just A . B. Takes angle AND length into account. If vectors are normalized (length = 1) then dot product = cosine, and it is the fastest to compute. Many models output normalized vectors.

3) Euclidean (L2) distance

Straight-line distance between the two points: sqrt of the sum of (a - b) squared. Smaller = more similar (opposite of cosine). For normalized vectors it gives the same ranking as cosine.

Metric	Higher means	Range	Use when
Cosine similarity	more similar	-1 to 1	default for text embeddings
Dot product	more similar	any	normalized vectors; fast; some models trained for it
Euclidean (L2)	LESS similar (it is a distance)	0 to infinity	image / numeric features

Tip: normalize once, then use dot product

If you scale every vector to length 1 when storing it, then cosine = dot product, and the database can use the cheapest, fastest maths. In sentence-transformers: *model.encode(texts, normalize_embeddings=True)*.

Also: some DBs (Chroma) report distance = 1 - cosine, so SMALLER is better there. Always check the docs!

```python
import numpy as np

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(cosine(vecs[0], vecs[1]))   # cat vs kitten   -> high
print(cosine(vecs[0], vecs[2]))   # cat vs stocks   -> low (near 0)

# built-in helper
from sentence_transformers import util
print(util.cos_sim(vecs[0], vecs[1]))
```

Choosing an Embedding Model

The embedding model is the most important choice for retrieval quality. Check the MTEB leaderboard (huggingface.co/spaces/mteb/leaderboard) because the best models change every few months - verify before you commit.

Model (examples)	Dims	Notes
all-MiniLM-L6-v2	384	tiny, fast, free, local; English; short inputs (~256 tokens)
bge-base-en-v1.5 / e5 family	768	strong free English models; e5 wants "query: " / "passage: " prefixes
BGE-M3 / multilingual-e5	1024	multilingual (Hindi, Marathi, etc.); BGE-M3 reads up to 8k tokens
OpenAI text-embedding-3-small / -large	1536 / 3072	API; can shorten with the *dimensions* option
Voyage AI, Cohere embed	varies	API; strong quality; Voyage AI is the provider Anthropic points to for embeddings

How to decide

- Language: Hindi / Marathi / mixed text? pick a multilingual model.
- Max input length: text longer than the limit is silently cut! Match it with your chunk size.
- Dimensions: more = better nuance but more RAM, disk, slower search (see the maths below).

- Local vs API: private data or zero cost → local model. Easiest start / top quality → API.
- Domain: legal / medical / code text may need a domain model, or fine-tuning.
- Test on YOUR data: take 30 real questions, check whether the right chunk is in the top 5.

Memory maths (why dimensions matter)

One number = float32 = 4 bytes. One vector of 1536 dims = 6,144 bytes. For 1 million chunks:

> 1,000,000 x 1536 x 4 bytes ~ 6.1 GB (384 dims -> 1.5 GB)

And a brute-force search does 1 million x 1536 multiplications per query. This is why vector DBs use smarter indexes (chapter 9). Compression tricks: Matryoshka embeddings (cut the vector to its first N dims), quantization (int8 / binary).

Code: API embedding + query/document prefixes

```python
# --- API model (OpenAI example) ---
from openai import OpenAI
client = OpenAI()                       # reads OPENAI_API_KEY
resp = client.embeddings.create(
    model="text-embedding-3-small",
    input=["hello world", "namaste duniya"],
)
vec = resp.data[0].embedding            # list of 1536 floats

# --- local model that needs prefixes (e5 style) ---
from sentence_transformers import SentenceTransformer
m = SentenceTransformer("intfloat/e5-base-v2")
doc_vecs = m.encode(["passage: Leave policy is 24 days a year"],
                    normalize_embeddings=True)
q_vec = m.encode(["query: how many holidays do I get?"],
                 normalize_embeddings=True)
```

Hugging Face Hub: the free model store

Hugging Face (HF) Hub hosts hundreds of thousands of open models. You refer to a model by its id = *owner/name*, and the library downloads it once into a local cache (*~/.cache/huggingface*). Free, runs on your own machine, your data never leaves it. Most embedding models are

used through the *sentence-transformers* library.

HF model id	Dims	Best for
sentence-transformers/all-MiniLM-L6-v2	384	fastest start, CPU friendly, English
sentence-transformers/all-mpnet-base-v2	768	better English quality than MiniLM
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	384	light multilingual (50+ languages)
BAAI/bge-small / base / large-en-v1.5	384 / 768 / 1024	strong English RAG; query instruction recommended
BAAI/bge-m3	1024	multilingual, long text (8k tokens), dense + sparse
intfloat/e5-base-v2, multilingual-e5-base	768	needs "query: " / "passage: " prefixes
nomic-ai/nomic-embed-text-v1.5	768	long context (8k); prefixes "search_query: " / "search_document: "
sentence-transformers/clip-ViT-B-32	512	images + text in one space

Always open the model card on the Hub: it tells you the dimensions, max length, required prefixes, pooling and license. Some models are gated (accept the license, then log in with a token: *huggingface-cli login*).

Rerankers and LLMs also live on the Hub

- Rerankers (cross-encoders): *BAAI/bge-reranker-base*, *BAAI/bge-reranker-v2-m3*, *cross-encoder/ms-marco-MiniLM-L-6-v2*.
- Open LLMs for generation: e.g. *Qwen/Qwen2.5-7B-Instruct*, *mistralai/Mistral-7B-Instruct-v0.3* (7B models need a GPU or quantization).
- Browse by task: "Sentence Similarity" / "Feature Extraction" for embeddings, "Text Generation" for LLMs.

```python
# 1) easiest: sentence-transformers downloads from the Hub
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-base-en-v1.5", device="cpu")  # "cuda" if GPU
q = model.encode(
    ["Represent this sentence for searching relevant passages: "
     "how many leave days?"], normalize_embeddings=True)
d = model.encode(["Employees get 24 days of paid leave."],
                 normalize_embeddings=True)
print(q @ d.T)                          # cosine similarity

# 2) plug into Chroma using the same HF id
from chromadb.utils import embedding_functions
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-base-en-v1.5")

# 3) plug into LangChain
# pip install langchain-huggingface
from langchain_huggingface import HuggingFaceEmbeddings
emb = HuggingFaceEmbeddings(
    model_name="BAAI/bge-base-en-v1.5",
    encode_kwargs={"normalize_embeddings": True})
```

Under the hood: raw transformers + mean pooling

This shows exactly what an embedding model does: tokens → one vector per token → average over real tokens (mask out padding) → normalize. (MiniLM / mpnet use mean pooling; BGE uses the first [CLS] token, so check the model card.)

```python
import torch, torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

name = "sentence-transformers/all-MiniLM-L6-v2"
tok = AutoTokenizer.from_pretrained(name)
mdl = AutoModel.from_pretrained(name).eval()

def embed(texts):
    batch = tok(texts, padding=True, truncation=True,
                max_length=256, return_tensors="pt")
    with torch.no_grad():
        hidden = mdl(**batch).last_hidden_state      # (B, tokens, 384)
    mask = batch["attention_mask"].unsqueeze(-1)     # 1 = real token
```

python (contd.)

```
    pooled = (hidden * mask).sum(1) / mask.sum(1)    # mean pooling
    return F.normalize(pooled, p=2, dim=1)           # length 1

v = embed(["I love pizza", "I enjoy pasta"])
print(v.shape, (v[0] @ v[1]).item())
```

No GPU? Use the HF Inference API

python

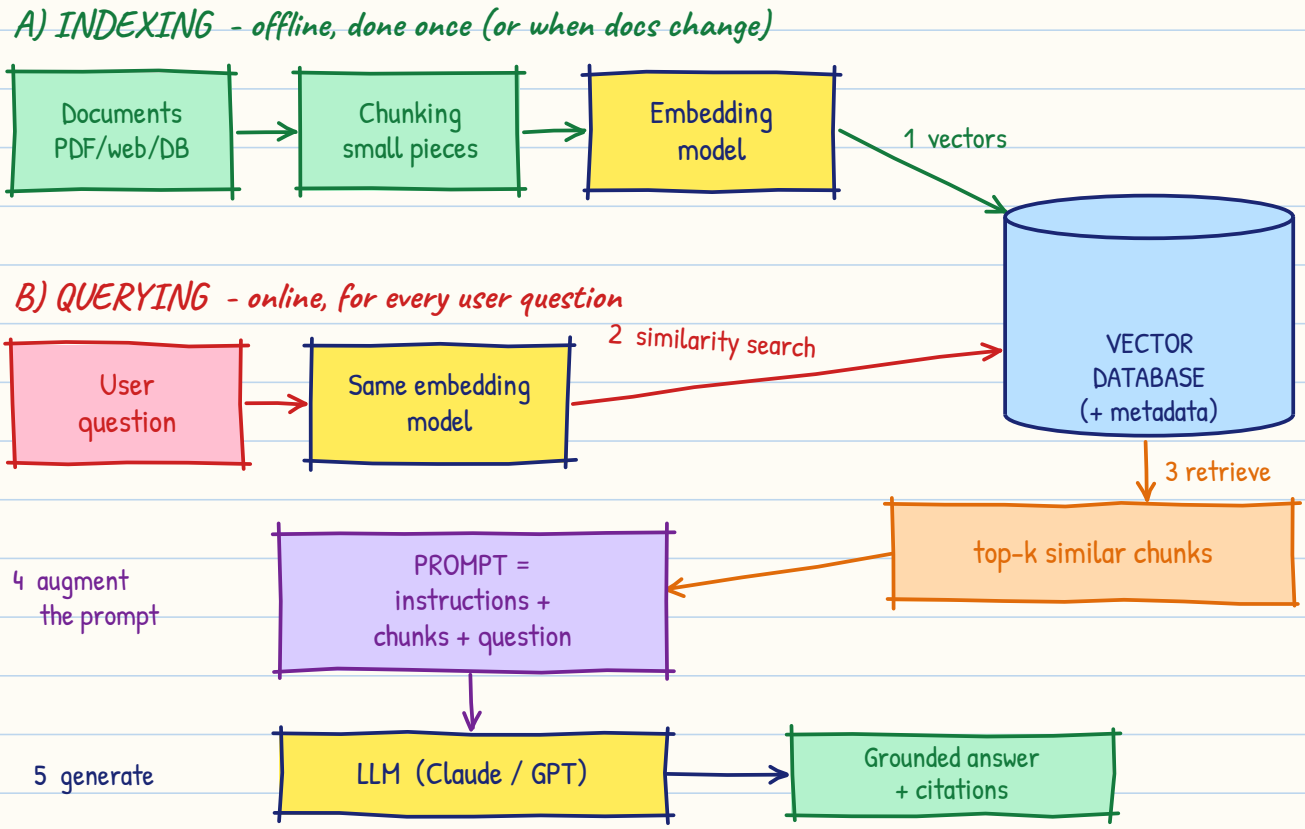
```
# pip install huggingface_hub      (create a token at huggingface.co/settings)
import os
from huggingface_hub import InferenceClient

client = InferenceClient(api_key=os.environ["HF_TOKEN"])
vec = client.feature_extraction("hello world",
                                model="BAAI/bge-base-en-v1.5")
print(vec.shape)      # numpy array of floats
```

Check on the model page that the model is served by an inference provider; free tiers are rate limited. Local models are better for private data; the hosted API is better for quick tests.

RAG Architecture, End to End

RAG has two phases. Phase A is done ahead of time; phase B runs on every question.



Phase A - Indexing (offline)

1. Load documents (PDF, HTML, Word, database rows, Notion ...) and clean the text.
2. Chunk them into small passages (next chapter).
3. Embed each chunk with the embedding model.
4. Store vector + original text + metadata (source, page, date, access level) in the vector DB.

Phase B - Querying (online)

1. Embed the question with the SAME model.
2. Search the vector DB for the top-k closest chunks (k is usually 3 to 10).
3. Augment: build a prompt = instructions + retrieved chunks + question.
4. Generate: the LLM answers using only that context and cites sources.

Why RAG and not just fine-tuning?

	RAG	Fine-tuning
Add new knowledge	just add documents	re-train the model
Cost / effort	low	high
Citations / sources	easy (we know the chunk)	hard
Hallucination control	good: "answer only from context"	weak
Per-user data access	filter by metadata	impossible
Best for	facts and documents	style, format, behaviour

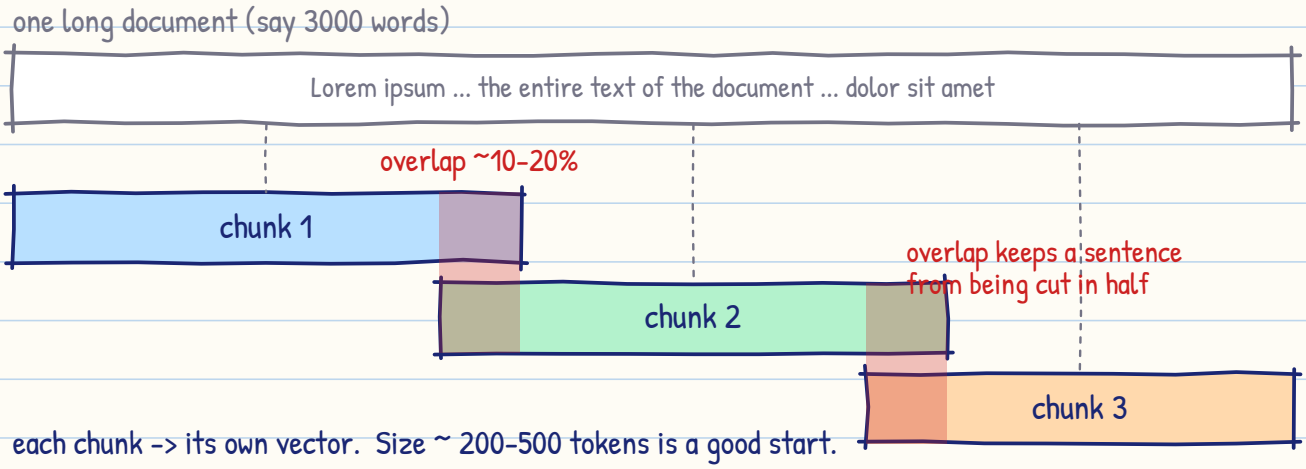
They combine well: fine-tune for behaviour, RAG for facts.

Prompt template that works

```
Answer the question using ONLY the context below.
If the answer is not in the context, say "I do not know".
Cite the chunk numbers like [1], [2].
Context:
[1] ...chunk text...
[2] ...chunk text...
Question: ...
```

Chunking: Cutting Documents Wisely

We cannot embed a whole 200-page PDF as one vector: it would blur 200 pages of topics into one average point, and the model has an input limit. So we cut text into chunks and embed each one. Chunk quality decides RAG quality - "garbage chunks in, garbage answers out".



each chunk -> its own vector. Size ~ 200-500 tokens is a good start.

Trade-off: small vs big chunks

	Small (100-200 tokens)	Big (800+ tokens)
Precision	high: focused meaning	low: many topics mixed
Context for LLM	may lack surrounding info	rich context
Cost	more vectors to store	fewer vectors, more prompt tokens

Good starting point: 300-500 tokens, 10-20% overlap, then tune using your evaluation set.

Chunking strategies

- Fixed size: every N characters/tokens. Simple, but may cut mid-sentence.
- Recursive (most popular): try to split on paragraphs, then lines, then sentences, then words - keeps natural units together.
- Structure-aware: split by headings / sections / markdown / HTML tags / code functions. Best for manuals, docs, code.
- Semantic: embed sentences, start a new chunk where the topic changes (similarity drops). Smarter, slower.
- Parent-child: search small chunks (precise), but send the bigger parent section to the LLM (context).

Always attach metadata

Store with each chunk: *source* file, *page*, *section title*, *date*, *author*, *language*, *access_level*. Metadata gives you citations, filters ("only 2024 policies", "only HR docs") and security (user only sees what they may see). Tip: prepend the section title to the chunk text - it greatly

helps retrieval.

```python
# 1) simplest chunker from scratch
def chunk_text(text, size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start + size])
        start += size - overlap          # step back by "overlap"
    return chunks

# 2) recommended: recursive splitter
# pip install langchain-text-splitters
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,                      # characters (not tokens) here
    chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_text(long_text)
print(len(chunks), chunks[0][:100])
```

PDF reality check

- Extraction quality matters: scanned PDFs need OCR, tables and multi-column layouts often break.
- Remove headers, footers, page numbers; they pollute every chunk.
- Look at 20 random chunks with your own eyes before embedding everything!

Vector Databases: Why and What

Problem: brute-force search is too slow

Simplest search: compare the query vector with EVERY stored vector and pick the best. This is exact (k-NN, k-nearest-neighbours) but the cost grows linearly. With 10 million chunks x 1536 dims, that is ~15 billion multiplications per query. Too slow and too memory hungry for real apps.

```python
import numpy as np

docs = ["Leave policy: 24 days per year", "Office wifi password rules",
        "How to claim travel expenses", "Work from home guidelines"]
doc_vecs = model.encode(docs, normalize_embeddings=True)      # (N, 384)

def search(query, k=2):
    q = model.encode([query], normalize_embeddings=True)[0]
    scores = doc_vecs @ q                    # cosine for ALL docs at once
    top = np.argsort(-scores)[:k]            # indexes of the best k
    return [(docs[i], round(float(scores[i]), 3)) for i in top]

print(search("how many holidays do I get?"))
```

Great for a few thousand chunks (a NumPy array is a perfectly fine "vector DB" for a prototype!). Beyond that we need something smarter.

What is a vector database?

A database designed to store vectors with their data and to find approximate nearest neighbours (ANN) very fast - trading a tiny bit of accuracy for huge speed. Besides search it gives you what any real database does:

- CRUD: add, update, delete vectors (documents change!).
- ANN index: HNSW / IVF etc. so search takes milliseconds, not seconds.
- Metadata filtering: combine meaning search with conditions.
- Persistence, scaling, replication, backups, access control.

Anatomy of one record

id	vector	document (text)	metadata
"c_001"	[0.12, -0.55, ..., 0.03]	"Employees get 24 days leave..."	{source: "hr.pdf", page: 3, year: 2024}
"c_002"	[0.31, 0.08, ..., -0.4]	"Travel claims must be filed..."	{source: "finance.pdf", page: 7}

The vector is used for searching; the text and metadata are what you get back and show to the LLM / user.

The landscape

Tool	Type	Pick it when
FAISS	library (in-memory)	research, custom pipelines, max speed; no server/filters built in
Chroma	embedded / simple server	learning, prototypes, small apps - easiest start
Qdrant	open-source DB (Rust)	strong filtering, self-host or cloud, production
Pinecone	fully managed cloud	you don't want to run infrastructure
Weaviate	open-source DB	built-in hybrid search and modules
Milvus	open-source, distributed	very large scale (billions)
pgvector	PostgreSQL extension	you already use Postgres; one DB for everything
Elasticsearch / OpenSearch	search engine + vectors	you already run it; great hybrid keyword+vector

Which one should I use?

- Learning / demo: Chroma or plain NumPy.
- Already have Postgres and < ~few million vectors: pgvector.
- Production, need filters + speed: Qdrant, Weaviate, Pinecone, Milvus.

The concepts (embed, index, filter, top-k) are the same everywhere - switching later is easy if your code isolates the DB behind one small function.

How ANN Indexes Work

ANN = Approximate Nearest Neighbour. Instead of checking everything, build an index (a map) so a query only visits a small, promising part of the data. We accept finding the true top-10 ~95-99% of the time. Measure of that quality = recall.

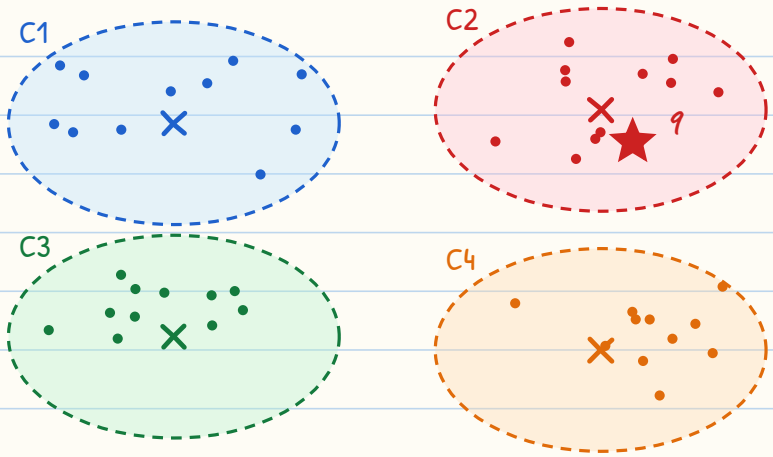
Flat index (exact)

No tricks: compare with every vector. 100% recall, slowest. Fine up to ~100k vectors, and used as the "ground truth" to test other indexes.

IVF - Inverted File (cluster first)

Idea: group the vectors into clusters using k-means. At query time, find the closest few cluster centres and search only inside those clusters. Like a supermarket: go to the right

aisle, not the whole store.



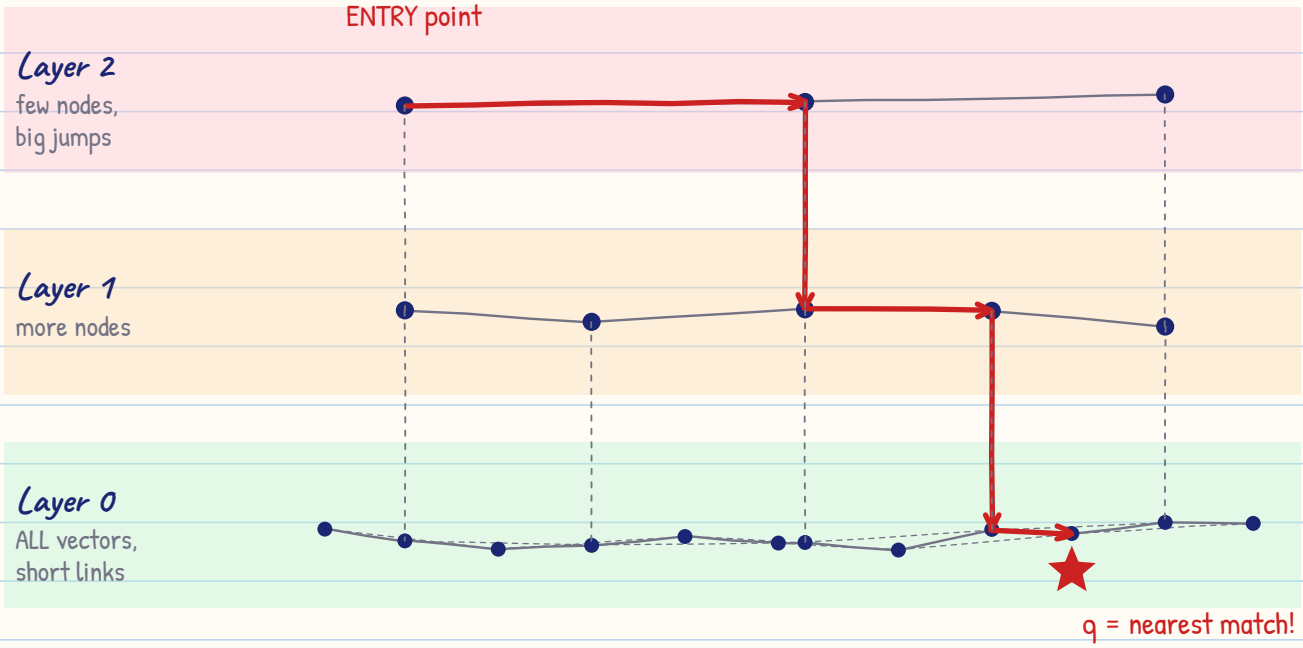
IVF in 3 steps

1. k-means makes N clusters
   (X = centroid)
2. query q -> nearest centroid
3. search ONLY those clusters
   (nprobe = how many)

nprobe up -> better recall,
but slower search.

HNSW - Hierarchical Navigable Small World (most popular)

Build a multi-layer graph. Each vector is a node linked to close neighbours. The top layer has few nodes and long links (like highways), lower layers get denser (like city roads), and layer 0 contains everything (like street level). Search starts at the top, greedily hops to whichever neighbour is closer to the query, then drops down a layer and repeats.



Because of the highway-to-street structure, search cost grows roughly with log(N) - millions of vectors in milliseconds.

HNSW knob	Meaning	Higher value ->
M	links per node	better recall, more memory
efConstruction	search width while BUILDING index	better graph, slower build
efSearch (ef)	search width while QUERYING	better recall, slower query

PQ - Product Quantization (compression)

Split each vector into sub-parts and replace each part with the id of its nearest "codebook" entry. A 1536-dim float vector (6 KB) can shrink to ~64-192 bytes - 30-100x smaller - with some accuracy loss. Often combined: IVF-PQ for billion-scale data on limited RAM.

Index	Speed	Recall	Memory	Best for
Flat	slow	100%	high	< 100k vectors, ground truth
IVF	fast	good	medium	millions; tune nprobe
HNSW	very fast	very good	high (graph)	default choice for most apps
IVF-PQ	fast	lower	very low	huge data, limited RAM

The eternal triangle

You can never maximise all three: Recall (accuracy) vs Speed vs Memory. Start with HNSW defaults, measure recall against a Flat index on a sample, then adjust *ef* / *nprobe*.

python

```python
# pip install faiss-cpu
import faiss, numpy as np
d = 384
X = doc_vecs.astype("float32")              # (N, 384), normalized
q = q_vec.astype("float32").reshape(1, -1)

# 1) Flat = exact, inner product (cosine for normalized vectors)
flat = faiss.IndexFlatIP(d)
flat.add(X)
scores, ids = flat.search(q, 3)             # top-3

# 2) HNSW = fast approximate
hnsw = faiss.IndexHNSWFlat(d, 32, faiss.METRIC_INNER_PRODUCT)   # M = 32
hnsw.hnsw.efConstruction = 200
hnsw.hnsw.efSearch = 64                     # raise for better recall
hnsw.add(X)
```

python (contd.)

```
scores, ids = hnsw.search(q, 3)

# 3) IVF = clusters
quantizer = faiss.IndexFlatIP(d)
ivf = faiss.IndexIVFFlat(quantizer, d, 100, faiss.METRIC_INNER_PRODUCT)
ivf.train(X)                                    # learn 100 centroids
ivf.add(X)
ivf.nprobe = 5                                  # clusters to visit
scores, ids = ivf.search(q, 3)
```

Hands-On: Chroma + Metadata Filters

Chroma runs inside your Python process (no server) and can embed text for you - the quickest way to build a working vector DB.

python

```python
# pip install chromadb sentence-transformers
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="./my_db")      # saved on disk
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2")

col = client.get_or_create_collection(
    name="company_docs",
    embedding_function=ef,                  # used for add AND query
    metadata={"hnsw:space": "cosine"},      # distance measure
)

col.upsert(                                 # upsert = add or update
    ids=["c1", "c2", "c3"],
    documents=[
        "Employees get 24 days of paid leave per year.",
        "Travel claims must be filed within 30 days.",
        "Work from home is allowed 2 days a week.",
    ],
    metadatas=[
        {"source": "hr.pdf", "year": 2024},
        {"source": "finance.pdf", "year": 2023},
        {"source": "hr.pdf", "year": 2024},
    ],
)
```

python

```python
res = col.query(
    query_texts=["how many holidays do I get?"],
    n_results=2,                            # top-k
    where={"source": "hr.pdf"},             # metadata filter
)
print(res["documents"][0])    # list of matching texts
print(res["distances"][0])    # smaller = closer (1 - cosine)
print(res["metadatas"][0])    # for citations

# other useful calls
col.get(ids=["c1"])                         # fetch by id
col.delete(ids=["c2"])                      # remove
col.count()                                 # how many records
# filter operators: {"year": {"$gte": 2024}}, {"$and": [...]}, {"$or": [...]}
```

Metadata filtering - why it is a superpower

- Security / multi-tenancy: *where={"user_id": "u42"}* so one customer never sees another's data.
- Freshness: only docs from this year; scope: only one product or one language.
- Pre-filter vs post-filter: pre-filtering narrows candidates before ANN search (accurate); post-filtering searches first then discards (can return fewer than k). Good DBs (Qdrant, Weaviate, pgvector...) handle this inside the index.

Idempotent ingestion

Give chunks stable ids (e.g. hash of file + chunk number) and use *upsert*. Re-running the ingestion script then updates instead of duplicating. When a source file changes, delete its old chunks by metadata and re-add.

Full RAG Pipeline in Code

Ingestion (phase A) + question answering (phase B) with Chroma and Claude. Replace the LLM call with any provider - the retrieval part is identical.

python

```python
# ---------- PHASE A: ingest ----------
from langchain_text_splitters import RecursiveCharacterTextSplitter
import hashlib

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=60)

def ingest(text, source):
    chunks = splitter.split_text(text)
    ids = [hashlib.md5(f"{source}-{i}".encode()).hexdigest()
           for i in range(len(chunks))]
    col.upsert(
        ids=ids,
        documents=chunks,
        metadatas=[{"source": source, "chunk": i}
                   for i in range(len(chunks))],
    )

ingest(open("hr_policy.txt").read(), "hr_policy.txt")
```

```python
# ---------- PHASE B: answer ----------
import anthropic
llm = anthropic.Anthropic()              # reads ANTHROPIC_API_KEY

def rag_answer(question, k=4):
    hits = col.query(query_texts=[question], n_results=k)
    chunks = hits["documents"][0]
    metas = hits["metadatas"][0]

    context = "\n\n".join(
        f"[{i+1}] ({m['source']}) {c}"
        for i, (c, m) in enumerate(zip(chunks, metas)))

    prompt = ("Answer using ONLY the context below. "
              "If the answer is not there, say 'I do not know'. "
              "Cite sources like [1].\n\n"
              f"Context:\n{context}\n\nQuestion: {question}")

    msg = llm.messages.create(
        model="claude-sonnet-5-5",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}],
    )
    return msg.content[0].text

print(rag_answer("How many days of paid leave do I get?"))
```

100% open-source version: Hugging Face LLM

Swap the Claude call for a local Hugging Face model and nothing leaves your machine. Retrieval code stays identical; only the generation step changes.

```python
# pip install transformers accelerate torch
from transformers import pipeline

gen = pipeline("text-generation",
               model="Qwen/Qwen2.5-7B-Instruct",   # any chat model from the Hub
               device_map="auto")                  # uses GPU if available

def rag_answer_hf(question, k=4):
    hits = col.query(query_texts=[question], n_results=k)
    context = "\n\n".join(f"[{i+1}] {c}"
                          for i, c in enumerate(hits["documents"][0]))
    prompt = ("Answer using ONLY the context. If missing, say 'I do not know'. "
              "Cite like [1].\n\nContext:\n" + context +
              "\n\nQuestion: " + question)
    out = gen([{"role": "user", "content": prompt}], max_new_tokens=300)
    return out[0]["generated_text"][-1]["content"]
```

Tip: 7B models need roughly 16 GB of GPU memory in half precision; use 4-bit quantization (bitsandbytes) or a smaller model if you have less.

Prompt-writing tips for RAG

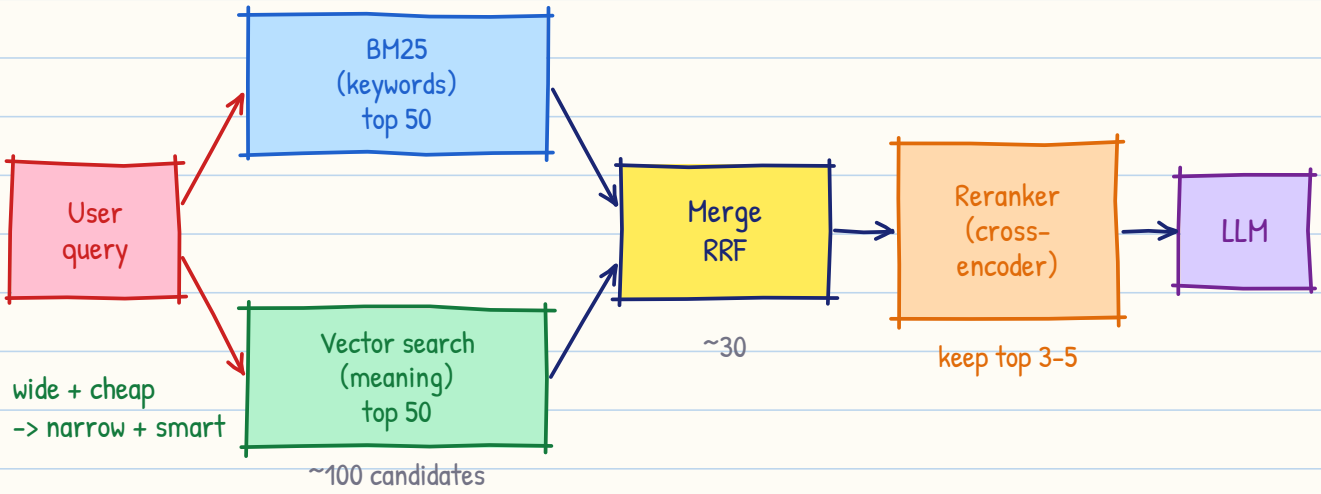
- Tell the model to use only the context and to say "I do not know" otherwise - this cuts hallucination.
- Number the chunks and ask for citations so users can verify.
- Put the question AFTER the context, and the most relevant chunk first or last (models can miss material in the middle of a long context - "lost in the middle").
- Treat retrieved text as data, not instructions (see prompt-injection warning in chapter 13).
- Log question, retrieved chunk ids, answer: essential for debugging and evaluation.

Making Retrieval Better

Basic RAG (embed, top-k, answer) is a great start but often retrieves wrong chunks. These upgrades give the biggest gains, roughly in this order:

1) Hybrid search = keywords + meaning

Vector search understands meaning but is weak on exact tokens: product codes ("SKU-4471"), names, error codes, acronyms. BM25 (classic keyword ranking) is the opposite. Run BOTH and merge the two ranked lists with Reciprocal Rank Fusion (RRF): each document earns 1/(60 + rank) from every list; add them up.




```python
# pip install rank_bm25
from rank_bm25 import BM25Okapi
bm25 = BM25Okapi([d.lower().split() for d in docs])

def hybrid_search(query, k=5):
    # keyword ranking
    kw_scores = bm25.get_scores(query.lower().split())
    kw_rank = list(np.argsort(-kw_scores)[:20])
    # vector ranking
    q = model.encode([query], normalize_embeddings=True)[0]
    vec_rank = list(np.argsort(-(doc_vecs @ q))[:20])
    return rrf([kw_rank, vec_rank])[:k]

def rrf(rank_lists, k=60):                # Reciprocal Rank Fusion
    scores = {}
    for ranking in rank_lists:
        for rank, doc_id in enumerate(ranking):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank + 1)
    return sorted(scores, key=scores.get, reverse=True)
```

2) Reranking - the biggest quality boost

Embedding search compares two pre-computed vectors (fast but rough: the query and chunk never "see" each other). A cross-encoder reranker reads the query and one chunk together and outputs a precise relevance score - much more accurate but too slow for the whole database. So: retrieve ~30-50 candidates cheaply, rerank, keep the best 3-5.

python

```python
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-base")       # or ms-marco-MiniLM

def retrieve_and_rerank(query, candidates, keep=4):
    pairs = [(query, c) for c in candidates]
    scores = reranker.predict(pairs)                    # one score per pair
    ranked = sorted(zip(scores, candidates),
                    key=lambda x: x[0], reverse=True)
    return [c for _, c in ranked[:keep]]
```

3) Query transformation

- Rewrite: users type vague / chatty text ("and what about the old one?"). Let an LLM turn it into a standalone search query using chat history.
- Multi-query: LLM writes 3 paraphrases; search all; merge results (higher recall).
- HyDE (Hypothetical Document Embeddings): LLM writes a fake answer, you embed THAT and search. A fake answer looks more like real chunks than a short question does.
- Decomposition: split "compare policy A and B" into two searches.

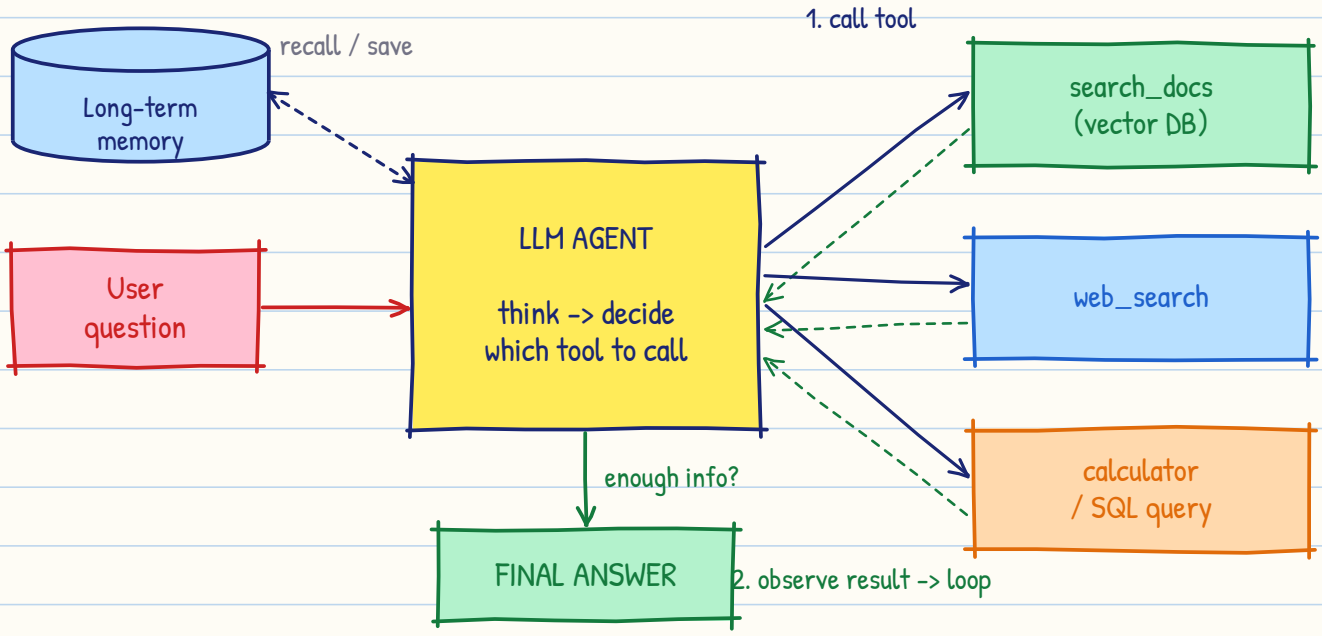
4) More tricks

- MMR (Maximal Marginal Relevance): pick results that are relevant AND different from each other, avoiding 5 near-duplicate chunks.
- Parent-document retrieval: match small chunks, return the larger section.
- Contextual chunks: prepend a one-line summary of the document / section to each chunk before embedding.
- Choosing k: retrieve 20-50, rerank down to 3-5. Too many chunks = noise, higher cost, and confused answers.

- Fine-tune the embedding model on your own (question, chunk) pairs when general models plateau.

Agentic RAG: Retrieval as a Tool

Classic RAG is a fixed pipeline: always retrieve once, then answer. Agentic RAG gives the LLM the retrieval system as a tool and lets it decide whether, what, and how many times to search - plus other tools - until it has enough to answer.



	Classic RAG	Agentic RAG
Flow	fixed: retrieve → generate	LLM decides, loops: think → act → observe
Searches	exactly one	zero, one or many (rephrases if results are poor)
Sources	one vector DB	many: docs DB, web, SQL, APIs, calculators
Hard questions	weak (multi-hop)	good: breaks the task into steps
Cost / latency	low, predictable	higher, variable - needs limits

Common agentic patterns

- Routing: decide which knowledge base (HR docs? product docs? SQL?) answers this question.
- Multi-hop: "Who manages the team that owns billing?" → search 1 finds the team, search 2 finds its manager.

- Self-check / corrective RAG: the agent grades retrieved chunks; if irrelevant, it rewrites the query and searches again, or falls back to web search.
- Agent memory: store past conversation summaries / user facts as vectors (metadata: *user_id*, *timestamp*) and retrieve them when relevant. The vector DB becomes the agent's long-term memory.

python

```
tools = [{
    "name": "search_docs",
    "description": "Search the company knowledge base. Returns the most "
                   "relevant passages. Call again with a different query "
                   "if the results do not answer the question.",
    "input_schema": {
        "type": "object",
        "properties": {"query": {"type": "string"}},
        "required": ["query"],
    },
}]

def search_docs(query):                    # our vector DB as a tool
    r = col.query(query_texts=[query], n_results=4)
    return "\n---\n".join(r["documents"][0])
```

```python
def run_agent(question, max_steps=6):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_steps):                    # safety limit!
        resp = llm.messages.create(
            model="claude-sonnet-5-5", max_tokens=1000,
            tools=tools, messages=messages)
        if resp.stop_reason != "tool_use":        # model is done
            return next(b.text for b in resp.content if b.type == "text")

        messages.append({"role": "assistant", "content": resp.content})
        results = []
        for block in resp.content:
            if block.type == "tool_use":
                out = search_docs(**block.input)    # run the tool
                results.append({"type": "tool_result",
                                "tool_use_id": block.id,
                                "content": out})
        messages.append({"role": "user", "content": results})
    return "Stopped: too many steps."
```

Safety for agents

- Prompt injection: a retrieved document may contain text like "ignore previous instructions". Tell the model that retrieved content is untrusted data, and never let it trigger sensitive actions without checks.
- Access control: filter by the USER's permissions in the search tool itself, not in the prompt.
- Limits: max steps, max tokens, timeouts, cost budget, logging of every tool call.

Evaluating Your RAG System

"It looks good in 3 demos" is not evaluation. Build a golden set: 30–100 real questions, each with the correct source chunk(s) and a reference answer. Re-run it after every change (chunk size, model, k, prompt).

Step 1: evaluate RETRIEVAL separately

Metric	Question it answers	Example
Hit rate / Recall@k	Is the right chunk anywhere in the top k?	42 of 50 questions → 84%
Precision@k	How many of the top k are relevant?	2 of 5 relevant → 0.4
MRR	How high is the first right chunk?	rank 1 → 1, rank 2 → 0.5, rank 4 → 0.25
nDCG	Are the best chunks ranked first?	rewards good order, not just presence

Step 2: evaluate GENERATION

- Faithfulness / groundedness: is every claim supported by the retrieved chunks?
- Answer relevance: does it actually answer the question asked?
- Context precision / recall: did we retrieve the right stuff and not too much junk?
- Tools: RAGAS, TruLens, DeepEval, LangSmith; or an "LLM-as-judge" prompt plus manual spot checks.

Debugging flowchart

Bad answer?

- Was the right chunk retrieved? NO → fix retrieval: chunking, embedding model, hybrid search, reranker, query rewriting, larger k.
- YES, but answer still wrong → fix generation: prompt, fewer / better-ordered chunks, stronger LLM.
- Answer fine but outdated → fix ingestion: re-index changed documents.

Mistakes, Checklist & Cheat Sheet

Top 10 mistakes

1. Different embedding models (or versions) for documents and queries.
2. Forgetting the model's max input length - long chunks silently truncated.
3. Chunks that are too big, too small, or cut mid-sentence; no overlap.
4. No metadata → no citations, no filters, no access control.
5. Using only vector search for IDs, names, codes - add BM25 / hybrid.
6. Sending 20 chunks to the LLM instead of reranking to the best 3-5.
7. Never evaluating retrieval separately from the LLM answer.
8. Not updating the index when documents change (stale answers).

9. Treating retrieved text as trusted instructions (prompt injection).
10. Over-engineering: start with simple RAG + good chunks, add complexity only when measurements demand it.

Production checklist

- Stable chunk ids, upsert-based ingestion, re-index on change, versioned embedding model name stored in metadata.
- Hybrid search + reranker; k tuned on the golden set.
- Metadata filters for tenant / permissions; logging of queries, chunks, answers, latency, cost.
- Fallback "I do not know" behaviour; user feedback buttons; regular evaluation runs.

Cheat sheet

Concept	One-line memory hook
Embedding	text → list of numbers; similar meaning = nearby numbers
Cosine similarity	angle between vectors; 1 = same direction, 0 = unrelated
Normalize	length 1 → dot product = cosine (fastest)
Chunking	300-500 tokens, 10-20% overlap, keep metadata
Vector DB	stores vector + text + metadata, finds nearest fast (ANN)
HNSW	layered graph: highways on top, streets below; tune M, ef
IVF	cluster first, search only nearest clusters (nprobe)
PQ	compress vectors 30-100x, lose a bit of accuracy
RAG	retrieve chunks → stuff in prompt → LLM answers with citations
Hybrid + rerank	BM25 + vectors merged by RRF, then cross-encoder picks top 3-5
Agentic RAG	retrieval is a tool; LLM loops think → search → observe
Evaluate	golden set; recall@k for retrieval, faithfulness for answers

Practice projects (in this order)

1. NumPy semantic search over 50 FAQ lines (no database).
2. Chat with your own PDF using Chroma + Claude (chapter 11 code).
3. Add hybrid search + reranker and measure recall@5 before / after.
4. Turn it into an agent with 2 tools (docs search + calculator or SQL).
5. Deploy with Qdrant or pgvector, add auth filters and an evaluation dashboard.

Happy building! - keep chunking carefully, always measure.